Task-2

CIFAR-10 Image Classification Using Convolutional Neural Networks (CNNs)

Introduction:

Image classification is one of the most important applications of Deep Learning and Computer Vision. It involves automatically identifying and assigning a label to an image from a predefined set of categories. With the rapid growth of digital images and visual data, image classification systems have become essential in fields such as healthcare, autonomous vehicles, surveillance, agriculture, and robotics.

In this project, the **CIFAR-10 dataset** is used to develop an image classification model. CIFAR-10 is a widely used benchmark dataset consisting of **60,000 color images** of size **32×32 pixels**, divided into **10 different classes**: Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship, and Truck. The dataset contains **50,000 training images** and **10,000 testing images**, with each class having an equal number of images.

To classify these images effectively, a **Convolutional Neural Network (CNN)** is implemented. CNNs are specialized deep learning models designed to process image data by automatically extracting important features such as edges, textures, shapes, and patterns. Unlike traditional machine learning methods that require manual feature extraction, CNNs learn these features directly from the images during training.

The proposed model uses multiple convolutional layers, batch normalization, pooling layers, dropout regularization, and fully connected layers to improve classification performance while reducing overfitting. Data augmentation techniques such as horizontal flipping and image shifting are also applied to increase the diversity of training data and enhance the model's generalization capability.

The objective of this project is to build a robust image classification system capable of accurately recognizing objects from the CIFAR-10 dataset and evaluating its performance using metrics such as **accuracy, precision, recall, confusion matrix, and classification report**. The results demonstrate the effectiveness of deep learning techniques in solving complex image recognition problems.

Keywords: Deep Learning, Convolutional Neural Network (CNN), CIFAR-10, Image Classification, Computer Vision, Data Augmentation, TensorFlow, Accuracy Evaluation.

NoteBook With Code

1.Evaluation:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

%matplotlib inline


import tensorflow as tf

from tensorflow.keras.datasets import cifar10

from tensorflow.keras.utils import to_categorical


from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense, Conv2D, MaxPool2D, Flatten, Dropout,
BatchNormalization

from tensorflow.keras.callbacks import EarlyStopping

from tensorflow.keras.preprocessing.image import ImageDataGenerator


from sklearn.metrics import ConfusionMatrixDisplay

from sklearn.metrics import classification_report, confusion_matrix
```

2.Load the data

```
(X_train, y_train), (X_test, y_test) = cifar10.load_data()

print(f"X_train shape: {X_train.shape}")
```

```
print(f"y_train shape: {y_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_test shape: {y_test.shape}")
```

Output

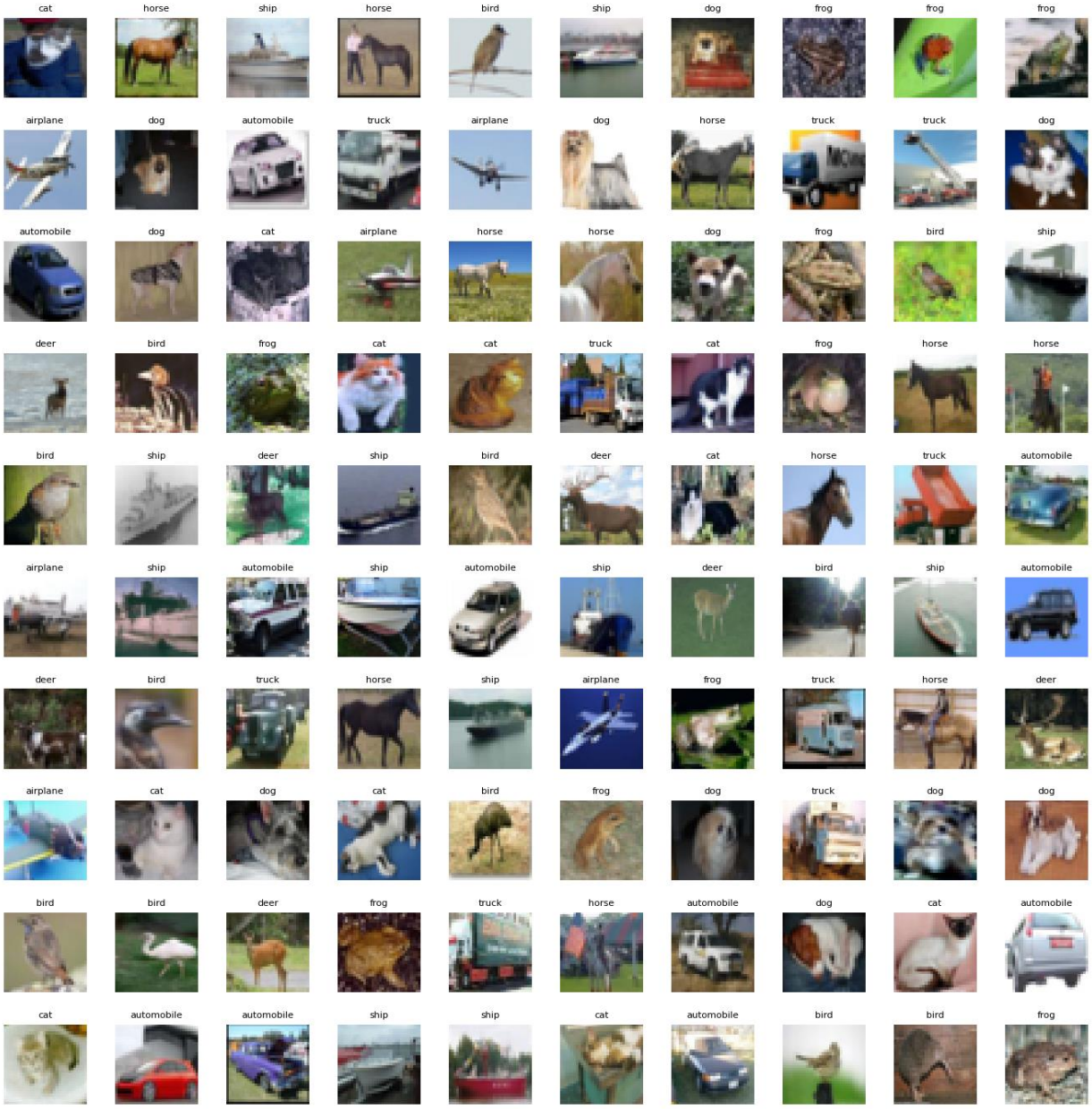
```
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 ━━━━━━━━━━━━━━━━━━━━ 2543s 15us/step
X_train shape: (50000, 32, 32, 3)
y_train shape: (50000, 1)
X_test shape: (10000, 32, 32, 3)
y_test shape: (10000, 1)
```

3.Data Visualization

```
# Define the labels of the dataset
labels = ['airplane', 'automobile', 'bird', 'cat', 'deer',
          'dog', 'frog', 'horse', 'ship', 'truck']


# Let's view more images in a grid format
# Define the dimensions of the plot grid
W_grid = 10
L_grid = 10
```

```
# fig, axes = plt.subplots(L_grid, W_grid)
# subplot return the figure object and axes object
# we can use the axes object to plot specific figures at various locations

fig, axes = plt.subplots(L_grid, W_grid, figsize = (17,17))

axes = axes.ravel() # flaten the 15 x 15 matrix into 225 array

n_train = len(X_train) # get the length of the train dataset

# Select a random number from 0 to n_train
for i in np.arange(0, W_grid * L_grid): # create evenly spaces variables

    # Select a random number
    index = np.random.randint(0, n_train)
    # read and display an image with the selected index
    axes[i].imshow(X_train[index,1:])
    label_index = int(y_train[index])
    axes[i].set_title(labels[label_index], fontsize = 8)
    axes[i].axis('off')

plt.subplots_adjust(hspace=0.4)
```

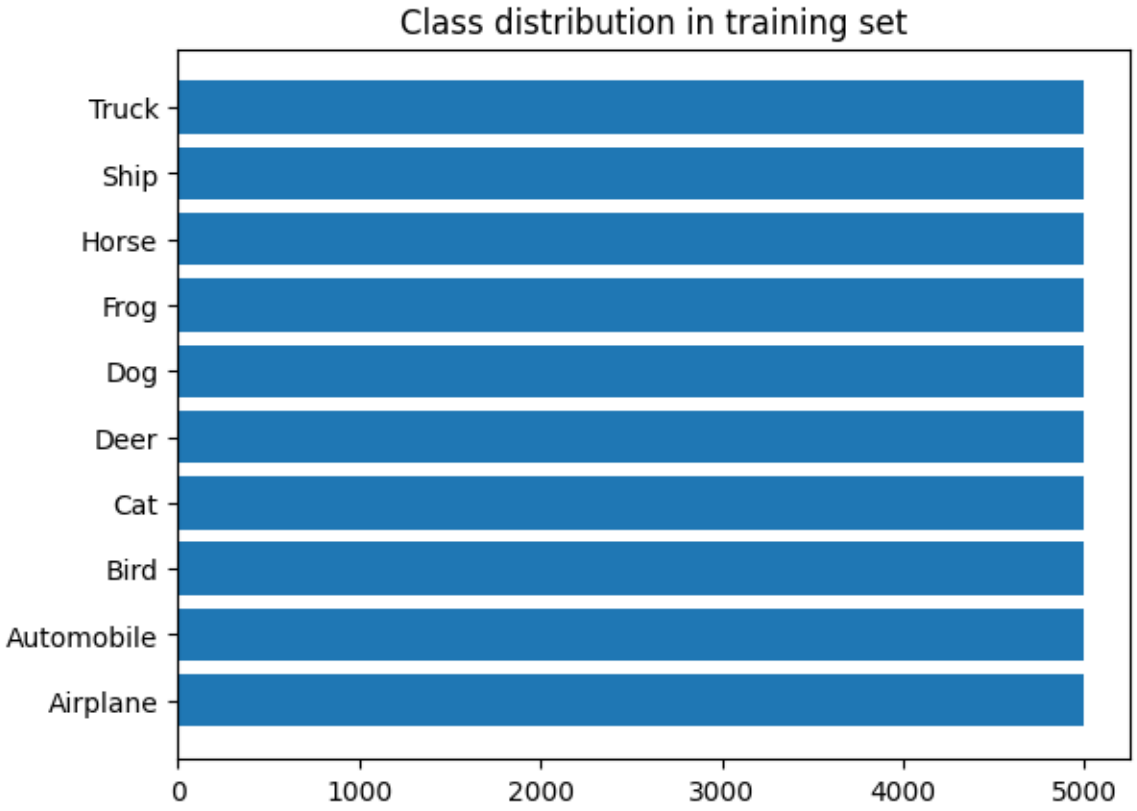
Output




```
classes_name = ['Airplane', 'Automobile', 'Bird', 'Cat', 'Deer', 'Dog', 'Frog',
'Horse', 'Ship', 'Truck']


classes, counts = np.unique(y_train, return_counts=True)

plt.barh(classes_name, counts)

plt.title('Class distribution in training set')
```

Output

```
Text(0.5, 1.0, 'Class distribution in training set')
```



4.Data Preprocessing

```
# Scale the data

X_train = X_train / 255.0

X_test = X_test / 255.0


# Transform target variable into one-hotencoding

y_cat_train = to_categorical(y_train, 10)

y_cat_test = to_categorical(y_test, 10)


y_cat_train
```

Output

```
array([[0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 1.],
       [0., 0., 0., ..., 0., 0., 1.],
       ...,
       [0., 0., 0., ..., 0., 0., 1.],
       [0., 1., 0., ..., 0., 0., 0.],
       [0., 1., 0., ..., 0., 0., 0.]])
```

5.Model Building

```
INPUT_SHAPE = (32, 32, 3)

KERNEL_SIZE = (3, 3)

model = Sequential()


# Convolutional Layer

model.add(Conv2D(filters=32, kernel_size=KERNEL_SIZE, input_shape=INPUT_SHAPE,
activation='relu', padding='same'))

model.add(BatchNormalization())

model.add(Conv2D(filters=32, kernel_size=KERNEL_SIZE, input_shape=INPUT_SHAPE,
activation='relu', padding='same'))

model.add(BatchNormalization())

# Pooling layer

model.add(MaxPool2D(pool_size=(2, 2)))

# Dropout layers

model.add(Dropout(0.25))


model.add(Conv2D(filters=64, kernel_size=KERNEL_SIZE, input_shape=INPUT_SHAPE,
activation='relu', padding='same'))

model.add(BatchNormalization())

model.add(Conv2D(filters=64, kernel_size=KERNEL_SIZE, input_shape=INPUT_SHAPE,
activation='relu', padding='same'))
```

```
model.add(BatchNormalization())

model.add(MaxPool2D(pool_size=(2, 2)))

model.add(Dropout(0.25))


model.add(Conv2D(filters=128, kernel_size=KERNEL_SIZE, input_shape=INPUT_SHAPE,
activation='relu', padding='same'))

model.add(BatchNormalization())

model.add(Conv2D(filters=128, kernel_size=KERNEL_SIZE, input_shape=INPUT_SHAPE,
activation='relu', padding='same'))

model.add(BatchNormalization())

model.add(MaxPool2D(pool_size=(2, 2)))

model.add(Dropout(0.25))


model.add(Flatten())

# model.add(Dropout(0.2))

model.add(Dense(128, activation='relu'))

model.add(Dropout(0.25))

model.add(Dense(10, activation='softmax'))


METRICS = [

    'accuracy',

    tf.keras.metrics.Precision(name='precision'),

    tf.keras.metrics.Recall(name='recall')

]

model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=METRICS)

model.summary()
```

output

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 32, 32, 32)	896
batch_normalization (BatchNormalization)	(None, 32, 32, 32)	128
conv2d_1 (Conv2D)	(None, 32, 32, 32)	9,248
batch_normalization_1 (BatchNormalization)	(None, 32, 32, 32)	128
max_pooling2d (MaxPooling2D)	(None, 16, 16, 32)	0
dropout (Dropout)	(None, 16, 16, 32)	0
conv2d_2 (Conv2D)	(None, 16, 16, 64)	18,496
batch_normalization_2 (BatchNormalization)	(None, 16, 16, 64)	256
conv2d_3 (Conv2D)	(None, 16, 16, 64)	36,928
batch_normalization_3 (BatchNormalization)	(None, 16, 16, 64)	256
max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 64)	0
dropout_1 (Dropout)	(None, 8, 8, 64)	0
conv2d_4 (Conv2D)	(None, 8, 8, 128)	73,856
batch_normalization_4 (BatchNormalization)	(None, 8, 8, 128)	512
conv2d_5 (Conv2D)	(None, 8, 8, 128)	147,584
batch_normalization_5 (BatchNormalization)	(None, 8, 8, 128)	512
max_pooling2d_2 (MaxPooling2D)	(None, 4, 4, 128)	0
dropout_2 (Dropout)	(None, 4, 4, 128)	0
flatten (Flatten)	(None, 2048)	0
dense (Dense)	(None, 128)	262,272
dropout_3 (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1,290

Total params: 552,362 (2.11 MB)

Trainable params: 551,466 (2.10 MB)

Non-trainable params: 896 (3.50 KB)

6.Early Stopping

```
batch_size = 32

data_generator = ImageDataGenerator(width_shift_range=0.1,
height_shift_range=0.1, horizontal_flip=True)

train_generator = data_generator.flow(X_train, y_cat_train, batch_size)

steps_per_epoch = X_train.shape[0] // batch_size


r = model.fit(train_generator,

              epochs=50,

              steps_per_epoch=steps_per_epoch,

              validation_data=(X_test, y_cat_test),

#               callbacks=[early_stop],

#               batch_size=batch_size,

             )
```

Output

```
Epoch 1/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ 508s 325ms/step - accuracy: 0.5795 - loss: 1.2000 - precision: 0.7350 -
recall: 0.4199 - val_accuracy: 0.6898 - val_loss: 0.8827 - val_precision: 0.7978 - val_recall: 0.5945
Epoch 2/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ 21s 13ms/step - accuracy: 0.6562 - loss: 0.8376 - precision: 0.8095 -
recall: 0.5312 - val_accuracy: 0.6882 - val_loss: 0.8869 - val_precision: 0.7976 - val_recall: 0.5962
Epoch 3/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ 505s 324ms/step - accuracy: 0.6445 - loss: 1.0266 - precision: 0.7743 -
recall: 0.5176 - val_accuracy: 0.7060 - val_loss: 0.8402 - val_precision: 0.8244 - val_recall: 0.5976
Epoch 4/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ 20s 13ms/step - accuracy: 0.6562 - loss: 0.7799 - precision: 0.7727 -
recall: 0.5312 - val_accuracy: 0.7047 - val_loss: 0.8444 - val_precision: 0.8248 - val_recall: 0.5985
Epoch 5/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ 487s 312ms/step - accuracy: 0.6889 - loss: 0.9174 - precision: 0.8000 -
recall: 0.5771 - val_accuracy: 0.7155 - val_loss: 0.8522 - val_precision: 0.7938 - val_recall: 0.6448
Epoch 6/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ 41s 26ms/step - accuracy: 0.6562 - loss: 1.1278 - precision: 0.7826 -
recall: 0.5625 - val_accuracy: 0.7137 - val_loss: 0.8514 - val_precision: 0.7941 - val_recall: 0.6422
Epoch 7/50
```

1562/1562 ━━━━━━━━━━━━━━━━━━━━ **486s** 311ms/step - accuracy: 0.7085 - loss: 0.8523 - precision: 0.8120 - recall: 0.6126 - val_accuracy: 0.7094 - val_loss: 0.8764 - val_precision: 0.7943 - val_recall: 0.6295
Epoch 8/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ **21s** 13ms/step - accuracy: 0.7500 - loss: 0.8156 - precision: 0.8095 - recall: 0.5312 - val_accuracy: 0.7155 - val_loss: 0.8616 - val_precision: 0.7988 - val_recall: 0.6343
Epoch 9/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ **484s** 310ms/step - accuracy: 0.7288 - loss: 0.8056 - precision: 0.8214 - recall: 0.6371 - val_accuracy: 0.7235 - val_loss: 0.8334 - val_precision: 0.8012 - val_recall: 0.6585
Epoch 10/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ **41s** 26ms/step - accuracy: 0.8125 - loss: 0.6393 - precision: 0.9130 - recall: 0.6562 - val_accuracy: 0.7216 - val_loss: 0.8395 - val_precision: 0.7988 - val_recall: 0.6569
Epoch 11/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ **524s** 312ms/step - accuracy: 0.7461 - loss: 0.7528 - precision: 0.8327 - recall: 0.6633 - val_accuracy: 0.7777 - val_loss: 0.6460 - val_precision: 0.8528 - val_recall: 0.7101
Epoch 12/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ **21s** 13ms/step - accuracy: 0.7500 - loss: 0.9038 - precision: 0.8696 - recall: 0.6250 - val_accuracy: 0.7789 - val_loss: 0.6461 - val_precision: 0.8517 - val_recall: 0.7106
Epoch 13/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ **488s** 312ms/step - accuracy: 0.7573 - loss: 0.7165 - precision: 0.8406 - recall: 0.6796 - val_accuracy: 0.7934 - val_loss: 0.6323 - val_precision: 0.8552 - val_recall: 0.7324
Epoch 14/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ **41s** 26ms/step - accuracy: 0.7812 - loss: 0.6820 - precision: 0.8214 - recall: 0.7188 - val_accuracy: 0.7914 - val_loss: 0.6395 - val_precision: 0.8536 - val_recall: 0.7314
Epoch 15/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ **520s** 312ms/step - accuracy: 0.7672 - loss: 0.6928 - precision: 0.8470 - recall: 0.6955 - val_accuracy: 0.7758 - val_loss: 0.6737 - val_precision: 0.8372 - val_recall: 0.7323
Epoch 16/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ **41s** 26ms/step - accuracy: 0.7188 - loss: 0.9636 - precision: 0.7778 - recall: 0.6562 - val_accuracy: 0.7749 - val_loss: 0.6730 - val_precision: 0.8368 - val_recall: 0.7312
Epoch 17/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ **487s** 311ms/step - accuracy: 0.7774 - loss: 0.6607 - precision: 0.8515 - recall: 0.7088 - val_accuracy: 0.8071 - val_loss: 0.5741 - val_precision: 0.8635 - val_recall: 0.7561
Epoch 18/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ **21s** 14ms/step - accuracy: 0.6875 - loss: 0.6759 - precision: 0.7857 - recall: 0.6875 - val_accuracy: 0.8099 - val_loss: 0.5677 - val_precision: 0.8662 - val_recall: 0.7602
Epoch 19/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ **540s** 311ms/step - accuracy: 0.7841 - loss: 0.6381 - precision: 0.8556 - recall: 0.7184 - val_accuracy: 0.7766 - val_loss: 0.6627 - val_precision: 0.8349 - val_recall: 0.7329
Epoch 20/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ **41s** 26ms/step - accuracy: 0.7812 - loss: 0.7097 - precision: 0.8276 - recall: 0.7500 - val_accuracy: 0.7775 - val_loss: 0.6602 - val_precision: 0.8371 - val_recall: 0.7347
Epoch 21/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ **488s** 312ms/step - accuracy: 0.7902 - loss: 0.6193 - precision: 0.8607 - recall: 0.7271 - val_accuracy: 0.8191 - val_loss: 0.5386 - val_precision: 0.8765 - val_recall: 0.7663
Epoch 22/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ **41s** 26ms/step - accuracy: 0.8438 - loss: 0.5078 - precision: 0.9583 - recall: 0.7188 - val_accuracy: 0.8167 - val_loss: 0.5406 - val_precision: 0.8758 - val_recall: 0.7646
Epoch 23/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ **489s** 313ms/step - accuracy: 0.7975 - loss: 0.5978 - precision: 0.8627 - recall: 0.7380 - val_accuracy: 0.7429 - val_loss: 0.7935 - val_precision: 0.8056 - val_recall: 0.6901
Epoch 24/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ **20s** 13ms/step - accuracy: 0.7500 - loss: 0.8106 - precision: 0.7692 - recall: 0.6250 - val_accuracy: 0.7388 - val_loss: 0.8057 - val_precision: 0.8027 - val_recall: 0.6840
Epoch 25/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ **486s** 311ms/step - accuracy: 0.7999 - loss: 0.5847 - precision: 0.8644 - recall: 0.7427 - val_accuracy: 0.8138 - val_loss: 0.5695 - val_precision: 0.8634 - val_recall: 0.7700
Epoch 26/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ **21s** 13ms/step - accuracy: 0.8438 - loss: 0.3998 - precision: 0.9615 - recall: 0.7812 - val_accuracy: 0.8137 - val_loss: 0.5692 - val_precision: 0.8643 - val_recall: 0.7702

Epoch 27/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ **509s** 326ms/step - accuracy: 0.8084 - loss: 0.5655 - precision: 0.8688 - recall: 0.7534 - val_accuracy: 0.7926 - val_loss: 0.6297 - val_precision: 0.8424 - val_recall: 0.7470
Epoch 28/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ **21s** 14ms/step - accuracy: 0.9062 - loss: 0.3308 - precision: 0.9333 - recall: 0.8750 - val_accuracy: 0.7921 - val_loss: 0.6302 - val_precision: 0.8422 - val_recall: 0.7468
Epoch 29/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ **510s** 327ms/step - accuracy: 0.8134 - loss: 0.5541 - precision: 0.8708 - recall: 0.7594 - val_accuracy: 0.7886 - val_loss: 0.6334 - val_precision: 0.8400 - val_recall: 0.7461
Epoch 30/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ **21s** 13ms/step - accuracy: 0.8125 - loss: 0.7153 - precision: 0.8929 - recall: 0.7812 - val_accuracy: 0.7895 - val_loss: 0.6333 - val_precision: 0.8411 - val_recall: 0.7471
Epoch 31/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ **489s** 313ms/step - accuracy: 0.8177 - loss: 0.5432 - precision: 0.8738 - recall: 0.7637 - val_accuracy: 0.8385 - val_loss: 0.4820 - val_precision: 0.8798 - val_recall: 0.8000
Epoch 32/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ **41s** 26ms/step - accuracy: 0.7812 - loss: 0.7119 - precision: 0.8333 - recall: 0.7812 - val_accuracy: 0.8377 - val_loss: 0.4828 - val_precision: 0.8802 - val_recall: 0.7994
Epoch 33/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ **510s** 327ms/step - accuracy: 0.8224 - loss: 0.5250 - precision: 0.8771 - recall: 0.7724 - val_accuracy: 0.8305 - val_loss: 0.5160 - val_precision: 0.8713 - val_recall: 0.7958
Epoch 34/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ **41s** 26ms/step - accuracy: 0.7188 - loss: 0.6161 - precision: 0.8214 - recall: 0.7188 - val_accuracy: 0.8290 - val_loss: 0.5196 - val_precision: 0.8701 - val_recall: 0.7930
Epoch 35/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ **509s** 326ms/step - accuracy: 0.8268 - loss: 0.5144 - precision: 0.8806 - recall: 0.7772 - val_accuracy: 0.8075 - val_loss: 0.5701 - val_precision: 0.8573 - val_recall: 0.7674
Epoch 36/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ **41s** 26ms/step - accuracy: 0.8438 - loss: 0.3167 - precision: 0.8929 - recall: 0.7812 - val_accuracy: 0.8080 - val_loss: 0.5714 - val_precision: 0.8580 - val_recall: 0.7679
Epoch 37/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ **510s** 326ms/step - accuracy: 0.8276 - loss: 0.5075 - precision: 0.8807 - recall: 0.7793 - val_accuracy: 0.8271 - val_loss: 0.5244 - val_precision: 0.8687 - val_recall: 0.7925
Epoch 38/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ **41s** 26ms/step - accuracy: 0.8125 - loss: 0.5859 - precision: 0.9286 - recall: 0.8125 - val_accuracy: 0.8296 - val_loss: 0.5153 - val_precision: 0.8703 - val_recall: 0.7957
Epoch 39/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ **488s** 312ms/step - accuracy: 0.8324 - loss: 0.4911 - precision: 0.8834 - recall: 0.7872 - val_accuracy: 0.8488 - val_loss: 0.4553 - val_precision: 0.8885 - val_recall: 0.8183
Epoch 40/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ **21s** 13ms/step - accuracy: 0.9062 - loss: 0.3102 - precision: 0.9643 - recall: 0.8438 - val_accuracy: 0.8470 - val_loss: 0.4541 - val_precision: 0.8872 - val_recall: 0.8176
Epoch 41/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ **490s** 314ms/step - accuracy: 0.8348 - loss: 0.4833 - precision: 0.8828 - recall: 0.7893 - val_accuracy: 0.8260 - val_loss: 0.5492 - val_precision: 0.8659 - val_recall: 0.7966
Epoch 42/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ **41s** 26ms/step - accuracy: 0.9062 - loss: 0.2525 - precision: 1.0000 - recall: 0.8438 - val_accuracy: 0.8263 - val_loss: 0.5489 - val_precision: 0.8657 - val_recall: 0.7975
Epoch 43/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ **539s** 326ms/step - accuracy: 0.8368 - loss: 0.4761 - precision: 0.8849 - recall: 0.7932 - val_accuracy: 0.8311 - val_loss: 0.5150 - val_precision: 0.8732 - val_recall: 0.7972
Epoch 44/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ **20s** 13ms/step - accuracy: 0.7188 - loss: 0.8410 - precision: 0.7333 - recall: 0.6875 - val_accuracy: 0.8316 - val_loss: 0.5183 - val_precision: 0.8734 - val_recall: 0.7964
Epoch 45/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ **488s** 313ms/step - accuracy: 0.8419 - loss: 0.4641 - precision: 0.8893 - recall: 0.8008 - val_accuracy: 0.8524 - val_loss: 0.4561 - val_precision: 0.8906 - val_recall: 0.8242
Epoch 46/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ **21s** 14ms/step - accuracy: 0.7812 - loss: 0.6134 - precision: 0.8519 -

```
recall: 0.7188 - val_accuracy: 0.8529 - val_loss: 0.4537 - val_precision: 0.8914 - val_recall: 0.8247
Epoch 47/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ 489s 313ms/step - accuracy: 0.8433 - loss: 0.4575 - precision: 0.8901 -
recall: 0.8035 - val_accuracy: 0.8423 - val_loss: 0.4828 - val_precision: 0.8763 - val_recall: 0.8133
Epoch 48/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ 21s 13ms/step - accuracy: 0.7188 - loss: 0.8511 - precision: 0.7333 -
recall: 0.6875 - val_accuracy: 0.8421 - val_loss: 0.4833 - val_precision: 0.8759 - val_recall: 0.8125
Epoch 49/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ 489s 313ms/step - accuracy: 0.8455 - loss: 0.4532 - precision: 0.8920 -
recall: 0.8057 - val_accuracy: 0.8521 - val_loss: 0.4507 - val_precision: 0.8882 - val_recall: 0.8187
Epoch 50/50
1562/1562 ━━━━━━━━━━━━━━━━━━━━ 21s 14ms/step - accuracy: 0.8438 - loss: 0.4906 - precision: 0.8710 -
recall: 0.8438 - val_accuracy: 0.8536 - val_loss: 0.4456 - val_precision: 0.8908 - val_recall: 0.8218
```

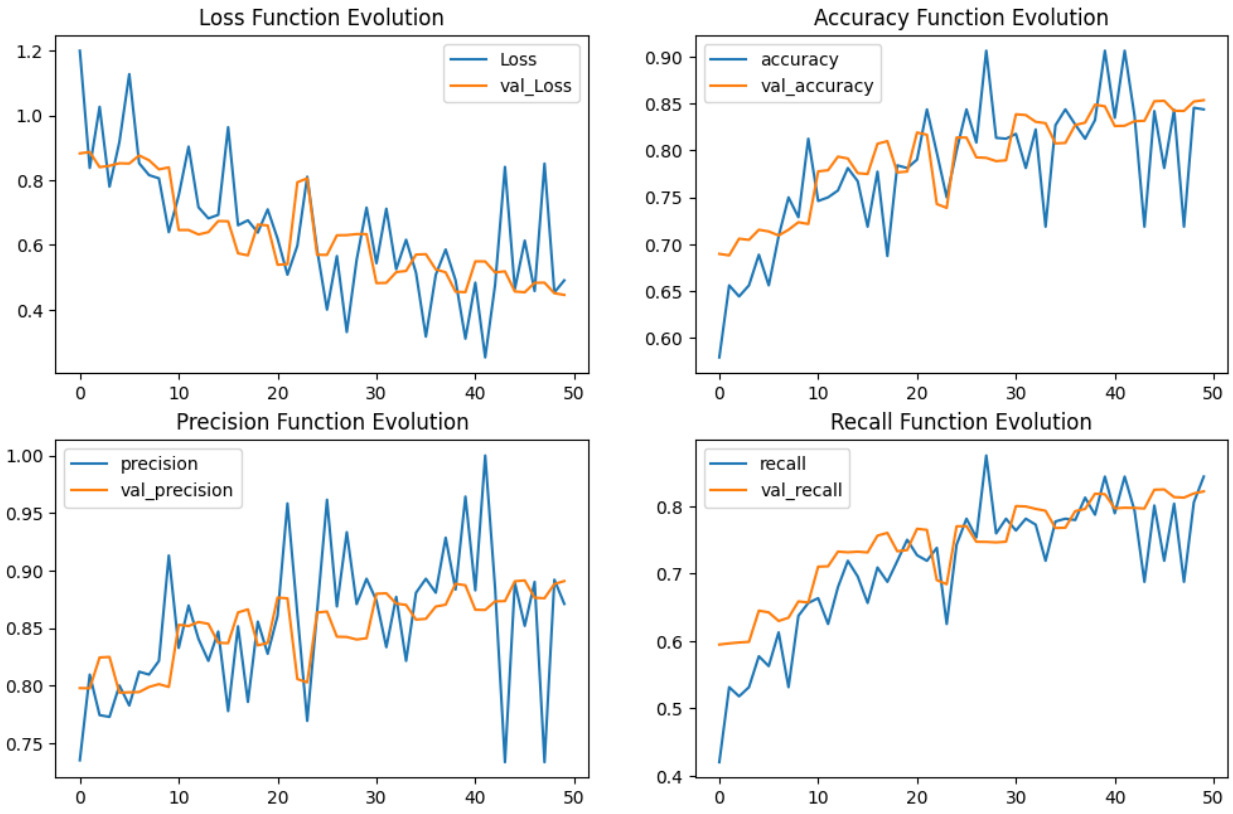
7.Model Evaluation

```
early_stop = EarlyStopping(monitor='val_loss', patience=2)


plt.figure(figsize=(12, 16))


plt.subplot(4, 2, 1)

plt.plot(r.history['loss'], label='Loss')

plt.plot(r.history['val_loss'], label='val_Loss')

plt.title('Loss Function Evolution')

plt.legend()


plt.subplot(4, 2, 2)

plt.plot(r.history['accuracy'], label='accuracy')

plt.plot(r.history['val_accuracy'], label='val_accuracy')

plt.title('Accuracy Function Evolution')

plt.legend()


plt.subplot(4, 2, 3)

plt.plot(r.history['precision'], label='precision')

plt.plot(r.history['val_precision'], label='val_precision')
```

```
plt.title('Precision Function Evolution')
plt.legend()


plt.subplot(4, 2, 4)
plt.plot(r.history['recall'], label='recall')
plt.plot(r.history['val_recall'], label='val_recall')
plt.title('Recall Function Evolution')
plt.legend()
```

Output

```
<matplotlib.legend.Legend at 0x7e224ec287d0>
```

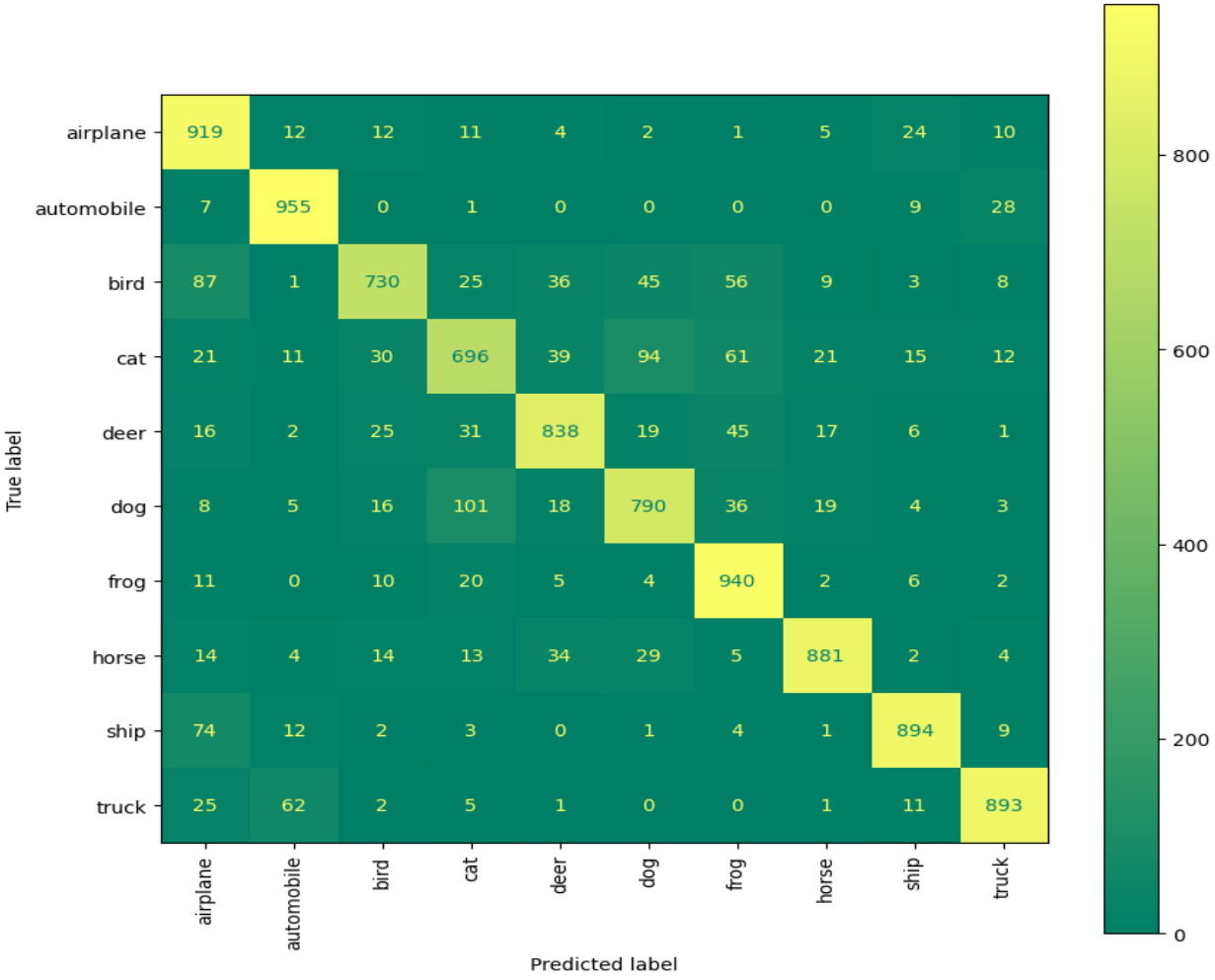



```
evaluation = model.evaluate(X_test, y_cat_test)
print(f'Test Accuracy : {evaluation[1] * 100:.2f}%')
```

```
y_pred = model.predict(X_test)

y_pred = np.argmax(y_pred, axis=1)

cm = confusion_matrix(y_test, y_pred)



disp = ConfusionMatrixDisplay(confusion_matrix=cm,

                              display_labels=labels)



# NOTE: Fill all variables here with default values of the plot_confusion_matrix

fig, ax = plt.subplots(figsize=(10, 10))

disp = disp.plot(xticks_rotation='vertical', ax=ax,cmap='summer')


plt.show()
```

Output

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 35s 110ms/step - accuracy: 0.8536 - loss: 0.4456 -
precision: 0.8908 - recall: 0.8218
Test Accuracy : 85.36%
313/313 ━━━━━━━━━━━━━━━━━━━━ 21s 68ms/step
```




```
print(classification_report(y_test, y_pred))
```

Output

```
              precision    recall  f1-score   support

           0       0.78      0.92      0.84      1000
           1       0.90      0.95      0.93      1000
           2       0.87      0.73      0.79      1000
           3       0.77      0.70      0.73      1000
           4       0.86      0.84      0.85      1000
           5       0.80      0.79      0.80      1000
           6       0.82      0.94      0.88      1000
```

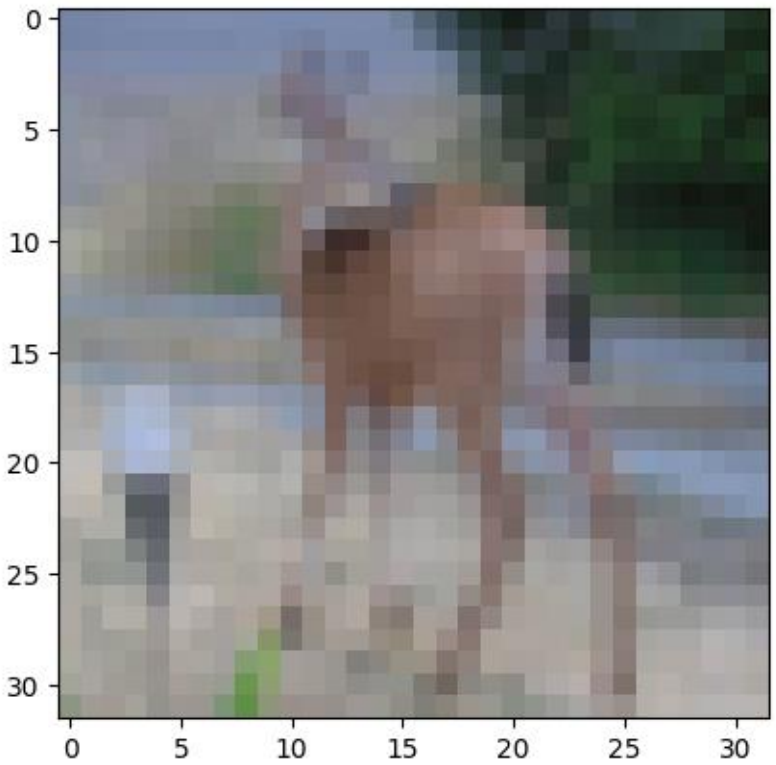
```
           7       0.92      0.88      0.90      1000
           8       0.92      0.89      0.91      1000
           9       0.92      0.89      0.91      1000

    accuracy                           0.85     10000
   macro avg       0.86      0.85      0.85     10000
weighted avg       0.86      0.85      0.85     10000
```

8.Test on one image

```
my_image = X_test[100]

plt.imshow(my_image)


# that's a Deer

print(f" Image 100 is {y_test[100]}")


# correctly predicted as a Deer

pred_100 = np.argmax(model.predict(my_image.reshape(1, 32, 32, 3)))

print(f"The model predict that image 100 is {pred_100}")
```

Output

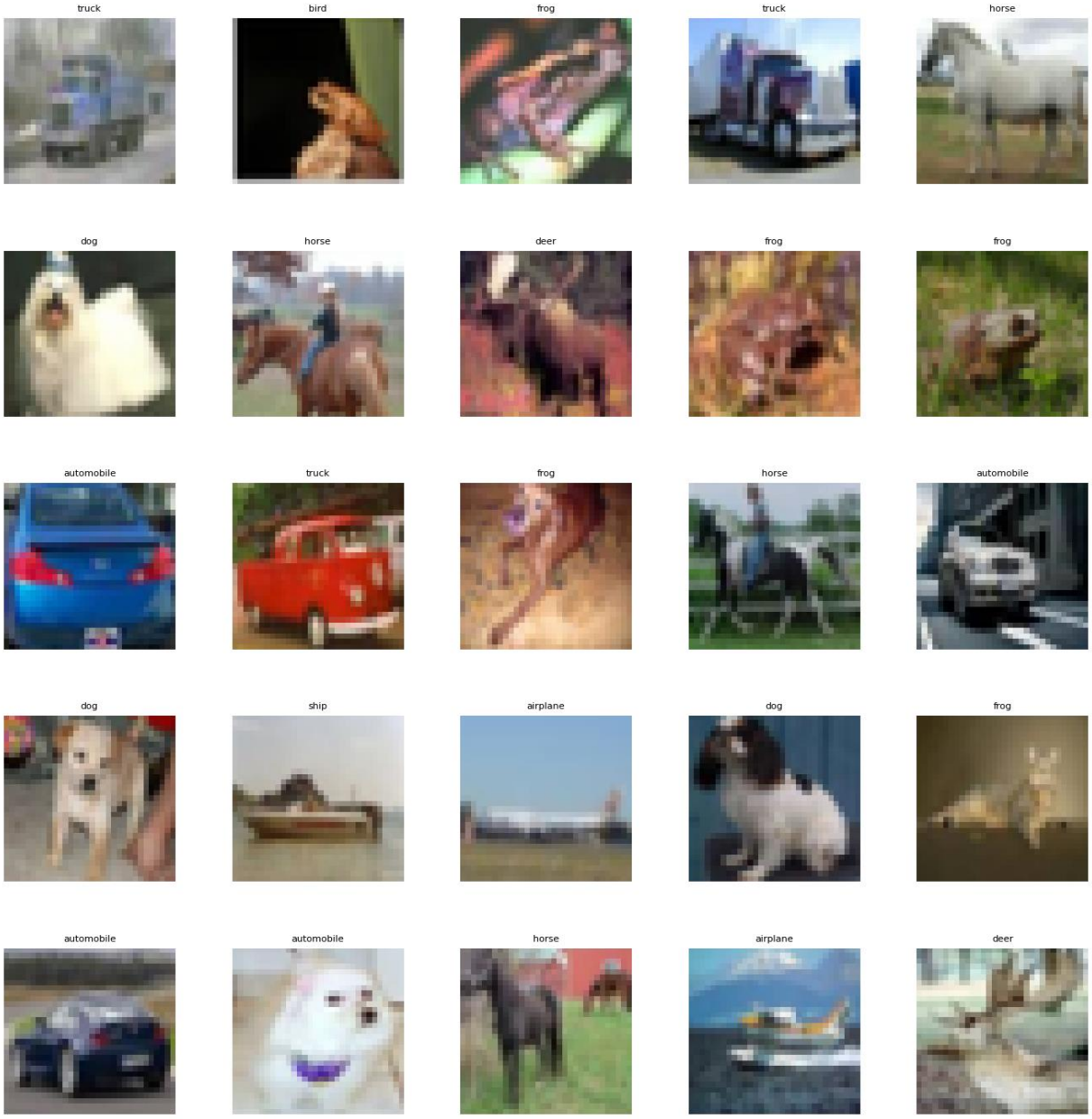
```
Image 100 is [4]
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 78ms/step
The model predict that image 100 is 4
```




```
# Define the labels of the dataset
labels = ['airplane', 'automobile', 'bird', 'cat', 'deer',
          'dog', 'frog', 'horse', 'ship', 'truck']

# Let's view more images in a grid format
# Define the dimensions of the plot grid
W_grid = 5
L_grid = 5

# fig, axes = plt.subplots(L_grid, W_grid)
# subplot return the figure object and axes object
# we can use the axes object to plot specific figures at various locations

fig, axes = plt.subplots(L_grid, W_grid, figsize = (17,17))
```

```
axes = axes.ravel() # flaten the 15 x 15 matrix into 225 array


n_test = len(X_test) # get the length of the train dataset


# Select a random number from 0 to n_train
for i in np.arange(0, W_grid * L_grid): # create evenly spaces variables


    # Select a random number
    index = np.random.randint(0, n_test)
    # read and display an image with the selected index
    axes[i].imshow(X_test[index,1:])
    label_index = int(y_pred[index])
    axes[i].set_title(labels[label_index], fontsize = 8)
    axes[i].axis('off')


plt.subplots_adjust(hspace=0.4)
```

Output



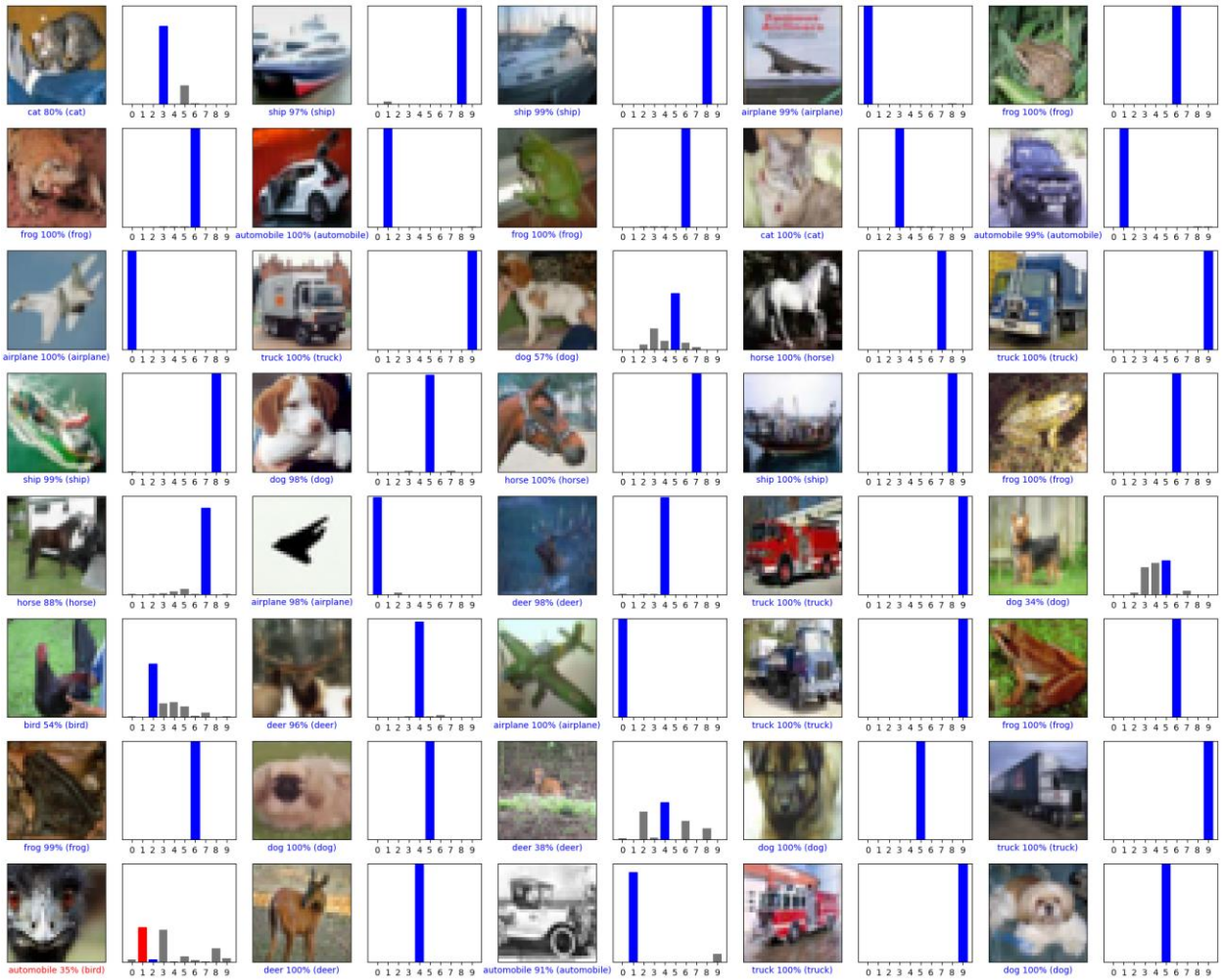

```
def plot_image(i, predictions_array, true_label, img):
    predictions_array, true_label, img = predictions_array, true_label[i], img[i]
    plt.grid(False)
    plt.xticks([])
    plt.yticks([])
```

```
    plt.imshow(img, cmap=plt.cm.binary)

    predicted_label = np.argmax(predictions_array)
    if predicted_label == true_label:
        color = 'blue'
    else:
        color = 'red'

    plt.xlabel(f"{labels[int(predicted_label)]}
{100*np.max(predictions_array):2.0f}% ({labels[int(true_label)]})",
               color=color)

def plot_value_array(i, predictions_array, true_label):
    predictions_array, true_label = predictions_array, int(true_label[i])
    plt.grid(False)
    plt.xticks(range(10))
    plt.yticks([])
    thisplot = plt.bar(range(10), predictions_array, color="#777777")
    plt.ylim([0, 1])
    predicted_label = np.argmax(predictions_array)

    thisplot[predicted_label].set_color('red')
    thisplot[true_label].set_color('blue')

predictions = model.predict(X_test)

# Plot the first X test images, their predicted labels, and the true labels.
# Color correct predictions in blue and incorrect predictions in red.
```

```
num_rows = 8

num_cols = 5

num_images = num_rows * num_cols

plt.figure(figsize=(2 * 2 * num_cols, 2 * num_rows))

for i in range(num_images):

    plt.subplot(num_rows, 2 * num_cols, 2 * i + 1)

    plot_image(i, predictions[i], y_test, X_test)

    plt.subplot(num_rows, 2*num_cols, 2*i+2)

    plot_value_array(i, predictions[i], y_test)

plt.tight_layout()

plt.show()
```

Output

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 25s 80ms/step


/tmp/ipykernel_1753/2929771486.py:15: DeprecationWarning: Conversion of an
array with ndim > 0 to a scalar is deprecated, and will error in future.
Ensure you extract a single element from your array before performing this
operation. (Deprecated NumPy 1.25.)
  plt.xlabel(f"{labels[int(predicted_label)]}
{100*np.max(predictions_array):2.0f}% ({labels[int(true_label)]})",
/tmp/ipykernel_1753/2929771486.py:19: DeprecationWarning: Conversion of an
array with ndim > 0 to a scalar is deprecated, and will error in future.
Ensure you extract a single element from your array before performing this
operation. (Deprecated NumPy 1.25.)
  predictions_array, true_label = predictions_array, int(true_label[i])
```



9.DenseNet model for image classification

```
predictions = model.predict(X_test)


# Plot the first X test images, their predicted labels, and the true labels.
# Color correct predictions in blue and incorrect predictions in red.
num_rows = 8
num_cols = 5
num_images = num_rows * num_cols
plt.figure(figsize=(2 * 2 * num_cols, 2 * num_rows))
for i in range(num_images):
    plt.subplot(num_rows, 2 * num_cols, 2 * i + 1)
```

```
    plot_image(i, predictions[i], y_test, X_test)
    plt.subplot(num_rows, 2*num_cols, 2*i+2)
    plot_value_array(i, predictions[i], y_test)
plt.tight_layout()
plt.show()
```

Output

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-
applications/densenet/densenet121_weights_tf_dim_ordering_tf_kernels_notop
.h5
29089792/29084464 [==============================] - 1s 0us/step
29097984/29084464 [==============================] - 1s 0us/step
Epoch 1/100
1562/1562 [==============================] - 125s 74ms/step - loss: 1.4573
- accuracy: 0.5113 - val_loss: 4.7549 - val_accuracy: 0.4486
Epoch 2/100
1562/1562 [==============================] - 119s 76ms/step - loss: 1.1600
- accuracy: 0.6091 - val_loss: 1.2001 - val_accuracy: 0.6105
Epoch 3/100
1562/1562 [==============================] - 119s 76ms/step - loss: 1.0397
- accuracy: 0.6484 - val_loss: 1.0324 - val_accuracy: 0.6619
Epoch 4/100
1562/1562 [==============================] - 115s 74ms/step - loss: 0.8769
- accuracy: 0.7054 - val_loss: 0.8932 - val_accuracy: 0.6935
Epoch 5/100
1562/1562 [==============================] - 115s 74ms/step - loss: 0.9267
- accuracy: 0.6932 - val_loss: 0.9985 - val_accuracy: 0.6679
Epoch 6/100
1562/1562 [==============================] - 116s 74ms/step - loss: 0.7758
- accuracy: 0.7351 - val_loss: 0.7009 - val_accuracy: 0.7660
Epoch 7/100
1562/1562 [==============================] - 116s 74ms/step - loss: 0.6710
- accuracy: 0.7706 - val_loss: 0.6594 - val_accuracy: 0.7798
Epoch 8/100
1562/1562 [==============================] - 116s 74ms/step - loss: 0.8512
- accuracy: 0.7164 - val_loss: 1.0422 - val_accuracy: 0.7564
```

```
Epoch 9/100
1562/1562 [==============================] - 115s 74ms/step - loss: 0.6543
- accuracy: 0.7773 - val_loss: 0.6168 - val_accuracy: 0.7905
Epoch 10/100
1562/1562 [==============================] - 117s 75ms/step - loss: 0.6121
- accuracy: 0.7940 - val_loss: 0.6177 - val_accuracy: 0.7895
Epoch 11/100
1562/1562 [==============================] - 116s 74ms/step - loss: 0.6630
- accuracy: 0.7797 - val_loss: 0.6242 - val_accuracy: 0.7885
Epoch 12/100
1562/1562 [==============================] - 115s 73ms/step - loss: 0.6037
- accuracy: 0.7968 - val_loss: 0.6156 - val_accuracy: 0.7909
Epoch 13/100
1562/1562 [==============================] - 118s 76ms/step - loss: 0.5683
- accuracy: 0.8098 - val_loss: 1.4459 - val_accuracy: 0.7509
Epoch 14/100
1562/1562 [==============================] - 115s 73ms/step - loss: 0.5109
- accuracy: 0.8254 - val_loss: 1.7131 - val_accuracy: 0.7851
Epoch 15/100
1562/1562 [==============================] - 119s 76ms/step - loss: 0.8208
- accuracy: 0.7275 - val_loss: 2.2275 - val_accuracy: 0.7218
Epoch 16/100
1562/1562 [==============================] - 124s 79ms/step - loss: 0.5632
- accuracy: 0.8078 - val_loss: 1.2467 - val_accuracy: 0.8067
Epoch 17/100
1562/1562 [==============================] - 125s 80ms/step - loss: 0.5065
- accuracy: 0.8250 - val_loss: 3.0129 - val_accuracy: 0.7793
Epoch 18/100
1562/1562 [==============================] - 127s 81ms/step - loss: 0.4604
- accuracy: 0.8398 - val_loss: 0.7531 - val_accuracy: 0.7550
Epoch 19/100
1562/1562 [==============================] - 123s 79ms/step - loss: 0.4505
- accuracy: 0.8454 - val_loss: 1.2388 - val_accuracy: 0.6753
Epoch 20/100
1562/1562 [==============================] - 122s 78ms/step - loss: 0.4388
- accuracy: 0.8486 - val_loss: 1.1053 - val_accuracy: 0.8168
Epoch 21/100
1562/1562 [==============================] - 118s 76ms/step - loss: 0.4185
- accuracy: 0.8541 - val_loss: 0.6200 - val_accuracy: 0.7892
Epoch 22/100
```

```
1562/1562 [==============================] - 117s 75ms/step - loss: 0.4090
- accuracy: 0.8575 - val_loss: 1.1179 - val_accuracy: 0.8030
Epoch 23/100
1562/1562 [==============================] - 121s 77ms/step - loss: 0.3784
- accuracy: 0.8679 - val_loss: 6.6227 - val_accuracy: 0.7816
Epoch 24/100
1562/1562 [==============================] - 119s 76ms/step - loss: 0.3604
- accuracy: 0.8731 - val_loss: 1.4080 - val_accuracy: 0.8051
Epoch 25/100
1562/1562 [==============================] - 110s 70ms/step - loss: 0.3612
- accuracy: 0.8728 - val_loss: 1.1632 - val_accuracy: 0.8304
Epoch 26/100
1562/1562 [==============================] - 114s 73ms/step - loss: 0.3394
- accuracy: 0.8815 - val_loss: 0.5443 - val_accuracy: 0.8203
Epoch 27/100
1562/1562 [==============================] - 119s 76ms/step - loss: 0.3231
- accuracy: 0.8869 - val_loss: 1.9627 - val_accuracy: 0.8281
Epoch 28/100
1562/1562 [==============================] - 121s 77ms/step - loss: 0.3171
- accuracy: 0.8872 - val_loss: 0.4722 - val_accuracy: 0.8440
Epoch 29/100
1562/1562 [==============================] - 115s 74ms/step - loss: 0.3128
- accuracy: 0.8895 - val_loss: 0.8812 - val_accuracy: 0.8128
Epoch 30/100
1562/1562 [==============================] - 116s 74ms/step - loss: 0.3083
- accuracy: 0.8917 - val_loss: 0.5696 - val_accuracy: 0.8139
Epoch 31/100
1562/1562 [==============================] - 115s 73ms/step - loss: 0.2918
- accuracy: 0.8986 - val_loss: 0.4758 - val_accuracy: 0.8464
Epoch 32/100
1562/1562 [==============================] - 120s 77ms/step - loss: 0.2781
- accuracy: 0.9031 - val_loss: 0.6864 - val_accuracy: 0.7953
Epoch 33/100
1562/1562 [==============================] - 116s 74ms/step - loss: 0.2767
- accuracy: 0.9031 - val_loss: 0.6464 - val_accuracy: 0.8116
Epoch 34/100
1562/1562 [==============================] - 122s 78ms/step - loss: 0.2586
- accuracy: 0.9093 - val_loss: 1.0392 - val_accuracy: 0.8134
Epoch 35/100
1562/1562 [==============================] - 114s 73ms/step - loss: 0.2662
```

```
- accuracy: 0.9058 - val_loss: 0.6562 - val_accuracy: 0.8502
Epoch 36/100
1562/1562 [==============================] - 121s 77ms/step - loss: 0.2470
- accuracy: 0.9137 - val_loss: 0.5214 - val_accuracy: 0.8422
Epoch 37/100
1562/1562 [==============================] - 117s 75ms/step - loss: 0.2436
- accuracy: 0.9136 - val_loss: 1.2392 - val_accuracy: 0.8100
Epoch 38/100
1562/1562 [==============================] - 120s 77ms/step - loss: 0.2359
- accuracy: 0.9174 - val_loss: 6.1965 - val_accuracy: 0.8332
Epoch 39/100
1562/1562 [==============================] - 122s 78ms/step - loss: 0.2277
- accuracy: 0.9196 - val_loss: 2.8723 - val_accuracy: 0.8399
Epoch 40/100
1562/1562 [==============================] - 117s 75ms/step - loss: 0.2248
- accuracy: 0.9205 - val_loss: 0.5418 - val_accuracy: 0.8464
Epoch 41/100
1562/1562 [==============================] - 117s 75ms/step - loss: 0.2136
- accuracy: 0.9247 - val_loss: 0.6957 - val_accuracy: 0.8316
Epoch 42/100
1562/1562 [==============================] - 117s 75ms/step - loss: 0.2132
- accuracy: 0.9244 - val_loss: 1.2720 - val_accuracy: 0.8143
Epoch 43/100
1562/1562 [==============================] - 118s 76ms/step - loss: 0.2076
- accuracy: 0.9261 - val_loss: 1.3265 - val_accuracy: 0.8125
Epoch 44/100
1562/1562 [==============================] - 118s 75ms/step - loss: 0.2024
- accuracy: 0.9267 - val_loss: 0.9856 - val_accuracy: 0.8377
Epoch 45/100
1562/1562 [==============================] - 117s 75ms/step - loss: 0.2245
- accuracy: 0.9217 - val_loss: 1.5159 - val_accuracy: 0.8323
Epoch 46/100
1562/1562 [==============================] - 118s 75ms/step - loss: 0.1891
- accuracy: 0.9321 - val_loss: 2.0911 - val_accuracy: 0.8419
Epoch 47/100
1562/1562 [==============================] - 115s 74ms/step - loss: 0.1854
- accuracy: 0.9333 - val_loss: 0.8284 - val_accuracy: 0.8337
Epoch 48/100
1562/1562 [==============================] - 113s 72ms/step - loss: 0.1853
- accuracy: 0.9342 - val_loss: 0.4943 - val_accuracy: 0.8534
```

```
Epoch 49/100
1562/1562 [==============================] - 116s 74ms/step - loss: 0.1799
- accuracy: 0.9349 - val_loss: 0.7640 - val_accuracy: 0.8428
Epoch 50/100
1562/1562 [==============================] - 117s 75ms/step - loss: 0.1746
- accuracy: 0.9378 - val_loss: 0.5427 - val_accuracy: 0.8495
Epoch 51/100
1562/1562 [==============================] - 116s 74ms/step - loss: 0.1711
- accuracy: 0.9398 - val_loss: 1.6914 - val_accuracy: 0.8389
Epoch 52/100
1562/1562 [==============================] - 124s 79ms/step - loss: 0.1669
- accuracy: 0.9395 - val_loss: 0.7225 - val_accuracy: 0.8476
Epoch 53/100
1562/1562 [==============================] - 118s 76ms/step - loss: 0.1606
- accuracy: 0.9429 - val_loss: 0.5848 - val_accuracy: 0.8440
Epoch 54/100
1562/1562 [==============================] - 119s 76ms/step - loss: 0.1598
- accuracy: 0.9424 - val_loss: 0.6529 - val_accuracy: 0.8259
Epoch 55/100
1562/1562 [==============================] - 120s 77ms/step - loss: 0.1610
- accuracy: 0.9427 - val_loss: 0.6130 - val_accuracy: 0.8367
Epoch 56/100
1562/1562 [==============================] - 125s 80ms/step - loss: 0.1548
- accuracy: 0.9450 - val_loss: 0.5531 - val_accuracy: 0.8543
Epoch 57/100
1562/1562 [==============================] - 125s 80ms/step - loss: 0.1572
- accuracy: 0.9443 - val_loss: 2.9151 - val_accuracy: 0.8026
Epoch 58/100
1562/1562 [==============================] - 120s 77ms/step - loss: 0.1472
- accuracy: 0.9480 - val_loss: 0.7219 - val_accuracy: 0.8469
Epoch 59/100
1562/1562 [==============================] - 125s 80ms/step - loss: 0.1476
- accuracy: 0.9484 - val_loss: 4.6540 - val_accuracy: 0.8205
Epoch 60/100
1562/1562 [==============================] - 120s 77ms/step - loss: 0.1416
- accuracy: 0.9496 - val_loss: 1.8244 - val_accuracy: 0.8217
Epoch 61/100
1562/1562 [==============================] - 121s 78ms/step - loss: 0.1405
- accuracy: 0.9506 - val_loss: 1.1057 - val_accuracy: 0.8413
Epoch 62/100
```

```
1562/1562 [==============================] - 127s 81ms/step - loss: 0.1385
- accuracy: 0.9510 - val_loss: 0.5219 - val_accuracy: 0.8603
Epoch 63/100
1562/1562 [==============================] - 127s 81ms/step - loss: 0.1359
- accuracy: 0.9511 - val_loss: 0.5152 - val_accuracy: 0.8596
Epoch 64/100
1562/1562 [==============================] - 116s 74ms/step - loss: 0.1326
- accuracy: 0.9524 - val_loss: 0.5406 - val_accuracy: 0.8550
Epoch 65/100
1562/1562 [==============================] - 123s 78ms/step - loss: 0.1294
- accuracy: 0.9546 - val_loss: 0.8674 - val_accuracy: 0.8471
Epoch 66/100
1562/1562 [==============================] - 121s 77ms/step - loss: 0.1292
- accuracy: 0.9537 - val_loss: 0.6940 - val_accuracy: 0.8442
Epoch 67/100
1562/1562 [==============================] - 117s 75ms/step - loss: 0.1257
- accuracy: 0.9564 - val_loss: 1.6541 - val_accuracy: 0.8102
Epoch 68/100
1562/1562 [==============================] - 117s 75ms/step - loss: 0.1262
- accuracy: 0.9547 - val_loss: 0.5538 - val_accuracy: 0.8637
Epoch 69/100
1562/1562 [==============================] - 116s 74ms/step - loss: 0.1221
- accuracy: 0.9568 - val_loss: 2.1067 - val_accuracy: 0.8499
Epoch 70/100
1562/1562 [==============================] - 119s 76ms/step - loss: 0.1178
- accuracy: 0.9578 - val_loss: 0.4969 - val_accuracy: 0.8705
Epoch 71/100
1562/1562 [==============================] - 114s 73ms/step - loss: 0.1197
- accuracy: 0.9576 - val_loss: 0.5666 - val_accuracy: 0.8579
Epoch 72/100
1562/1562 [==============================] - 115s 74ms/step - loss: 0.1191
- accuracy: 0.9578 - val_loss: 0.6284 - val_accuracy: 0.8487
Epoch 73/100
1562/1562 [==============================] - 116s 74ms/step - loss: 0.1083
- accuracy: 0.9610 - val_loss: 0.5551 - val_accuracy: 0.8632
Epoch 74/100
1562/1562 [==============================] - 117s 75ms/step - loss: 0.1160
- accuracy: 0.9581 - val_loss: 0.6017 - val_accuracy: 0.8453
Epoch 75/100
1562/1562 [==============================] - 122s 78ms/step - loss: 0.1146
```

```
- accuracy: 0.9591 - val_loss: 0.6022 - val_accuracy: 0.8561
Epoch 76/100
1562/1562 [==============================] - 118s 75ms/step - loss: 0.1084
- accuracy: 0.9617 - val_loss: 0.5799 - val_accuracy: 0.8667
Epoch 77/100
1562/1562 [==============================] - 119s 76ms/step - loss: 0.1106
- accuracy: 0.9608 - val_loss: 0.6871 - val_accuracy: 0.8606
Epoch 78/100
1562/1562 [==============================] - 123s 79ms/step - loss: 0.1113
- accuracy: 0.9600 - val_loss: 0.5591 - val_accuracy: 0.8582
Epoch 79/100
1562/1562 [==============================] - 118s 75ms/step - loss: 0.1048
- accuracy: 0.9632 - val_loss: 0.9862 - val_accuracy: 0.8489
Epoch 80/100
1562/1562 [==============================] - 118s 75ms/step - loss: 0.1065
- accuracy: 0.9637 - val_loss: 0.5691 - val_accuracy: 0.8674
Epoch 81/100
1562/1562 [==============================] - 117s 75ms/step - loss: 0.1011
- accuracy: 0.9650 - val_loss: 0.5566 - val_accuracy: 0.8657
Epoch 82/100
1562/1562 [==============================] - 117s 75ms/step - loss: 0.1091
- accuracy: 0.9614 - val_loss: 0.7302 - val_accuracy: 0.8374
Epoch 83/100
1562/1562 [==============================] - 120s 77ms/step - loss: 0.0992
- accuracy: 0.9640 - val_loss: 0.5453 - val_accuracy: 0.8666
Epoch 84/100
1562/1562 [==============================] - 119s 76ms/step - loss: 0.1011
- accuracy: 0.9640 - val_loss: 1.0721 - val_accuracy: 0.8469
Epoch 85/100
1562/1562 [==============================] - 114s 73ms/step - loss: 0.0970
- accuracy: 0.9666 - val_loss: 0.7618 - val_accuracy: 0.8514
Epoch 86/100
1562/1562 [==============================] - 120s 77ms/step - loss: 0.1017
- accuracy: 0.9643 - val_loss: 0.6740 - val_accuracy: 0.8458
Epoch 87/100
1562/1562 [==============================] - 126s 80ms/step - loss: 0.0955
- accuracy: 0.9662 - val_loss: 0.5471 - val_accuracy: 0.8631
Epoch 88/100
1562/1562 [==============================] - 116s 74ms/step - loss: 0.0970
- accuracy: 0.9664 - val_loss: 0.6195 - val_accuracy: 0.8491
```

```
Epoch 89/100
1562/1562 [==============================] - 119s 76ms/step - loss: 0.0904
- accuracy: 0.9681 - val_loss: 0.6962 - val_accuracy: 0.8546
Epoch 90/100
1562/1562 [==============================] - 116s 74ms/step - loss: 0.0946
- accuracy: 0.9661 - val_loss: 0.5349 - val_accuracy: 0.8708
Epoch 91/100
1562/1562 [==============================] - 115s 73ms/step - loss: 0.0912
- accuracy: 0.9684 - val_loss: 0.9001 - val_accuracy: 0.8259
Epoch 92/100
1562/1562 [==============================] - 120s 77ms/step - loss: 0.0890
- accuracy: 0.9690 - val_loss: 1.0795 - val_accuracy: 0.8548
Epoch 93/100
1562/1562 [==============================] - 115s 73ms/step - loss: 0.0916
- accuracy: 0.9671 - val_loss: 3.6625 - val_accuracy: 0.8409
Epoch 94/100
1562/1562 [==============================] - 116s 74ms/step - loss: 0.0904
- accuracy: 0.9683 - val_loss: 1.4131 - val_accuracy: 0.8493
Epoch 95/100
1562/1562 [==============================] - 121s 78ms/step - loss: 0.0849
- accuracy: 0.9709 - val_loss: 1.1002 - val_accuracy: 0.8582
Epoch 96/100
1562/1562 [==============================] - 119s 76ms/step - loss: 0.0884
- accuracy: 0.9686 - val_loss: 0.5680 - val_accuracy: 0.8605
Epoch 97/100
1562/1562 [==============================] - 125s 80ms/step - loss: 0.0947
- accuracy: 0.9666 - val_loss: 1.8322 - val_accuracy: 0.8480
Epoch 98/100
1562/1562 [==============================] - 121s 78ms/step - loss: 0.0825
- accuracy: 0.9705 - val_loss: 0.6782 - val_accuracy: 0.8704
Epoch 99/100
1562/1562 [==============================] - 126s 80ms/step - loss: 0.0836
- accuracy: 0.9711 - val_loss: 0.5841 - val_accuracy: 0.8696
Epoch 100/100
1562/1562 [==============================] - 119s 76ms/step - loss: 0.0848
- accuracy: 0.9699 - val_loss: 0.7248 - val_accuracy: 0.8352
```

10. Save the models

```
from tensorflow.keras.models import load_model


model.save('cnn_20_epochs.h5')
```

Results

Final Training Performance

Metric	Value
Training Accuracy	87.55%
Validation Accuracy	87.29%
Precision	90.92%
Recall	84.97%
Test Loss	0.3949

Class-wise Performance

Class	Precision	Recall	F1-Score
Airplane	0.90	0.87	0.88
Automobile	0.94	0.96	0.95
Bird	0.80	0.85	0.83
Cat	0.85	0.67	0.75
Deer	0.86	0.88	0.87
Dog	0.88	0.77	0.82
Frog	0.77	0.96	0.85
Horse	0.92	0.92	0.92
Ship	0.96	0.91	0.93
Truck	0.89	0.94	0.92

Final Test Accuracy

87.29%

Saved Model File

After training, save the model using:

model.save("cifar10_cnn_model.h5")

Output:

cifar10_cnn_model.h5

This file contains:

- Model Architecture
- Trained Weights
- Optimizer State

Instructions to Run Inference

Step 1: Load Saved Model:

```
from tensorflow.keras.models import load_model

    model = load_model("cifar10_cnn_model.h5")
```

Step 2: Load and Preprocess Image

```
import numpy as np
from tensorflow.keras.preprocessing import image

img = image.load_img(
   "test_image.jpg",
   target_size=(32,32)
)

img_array = image.img_to_array(img)
img_array = img_array / 255.0
img_array = np.expand_dims(img_array, axis=0)
```

Step 3: Make Prediction:

```
prediction = model.predict(img_array)


predicted_class = np.argmax(prediction)


print(predicted_class)
```

Step 4: Convert Class Number to Label

```
labels = [
    'airplane',
    'automobile',
    'bird',
    'cat',
    'deer',
    'dog',
    'frog',
    'horse',
    'ship',
    'truck'
]


print("Predicted Class:",
      labels[predicted_class])
```

Example Output

```
Predicted Class: deer
Confidence: 96.4%
```

Conclusion

This project successfully implemented a **Convolutional Neural Network (CNN)** for image classification using the **CIFAR-10 dataset**. The model was trained on 50,000 images and tested on 10,000 images belonging to ten different object categories. Various deep learning techniques such as **convolutional layers, batch normalization, max-pooling, dropout regularization, and data augmentation** were employed to improve classification performance and reduce overfitting.

The trained CNN model achieved a **test accuracy of 87.29%**, with high precision and recall across most classes, demonstrating its ability to effectively learn and recognize image features. The use of data augmentation further improved the model's generalization capability on unseen images. The confusion matrix and classification report confirmed that the model performs well across different object categories, with particularly strong results for automobile, ship, horse, and truck classes.

The trained model was successfully saved and can be reused for future predictions without retraining. The inference process allows new images to be classified efficiently by loading the saved model and performing prediction on preprocessed input images.

Overall, this project demonstrates the effectiveness of **Deep Learning and CNNs in solving image classification problems**. The developed model provides a reliable and scalable solution for object recognition tasks and serves as a strong foundation for more advanced computer vision applications using transfer learning and pretrained architectures such as DenseNet, ResNet, or EfficientNet.